UNIT 5 · TOPIC 5.3

Protecting Stored Data with Cryptography

Assignment

Instructions: 12 practice questions + 3 coding assignment(s), 27 points total. Show your work for short-answer items. Coding assignments may be completed here or in the interactive auto-graded environment on the course site (same problem, same points).

Name: **Date:** **Class:**

Practice Questions

5.3.a · Encryption vs. hashing

1. An engineer needs to store credit card numbers so they can be processed and charged at the end of every month. Which cryptographic primitive must they use? (1 pt)

- ○ A. Encryption, because the process must be reversible to retrieve the original card numbers
- ○ B. Hashing, because it protects the integrity of the credit card numbers
- ○ C. Encoding, because Base64 keeps card numbers safe on the disk
- ○ D. Obfuscation, because it hides details and is fully reversible without keys

2. What is the primary security goal of adding a salt to a password before hashing it? (1 pt)

- ○ A. To make the hashing function compute slower and consume more CPU
- ○ B. To ensure that identical passwords yield different digests, defeating precomputed lookup attacks
- ○ C. To encrypt the password so it can be decrypted by administrators
- ○ D. To detect if the password has been modified during transmission over the network

3. Select ALL of the following statements that represent common security misconceptions. (3 pts)

- □ A. Encoding data in Base64 provides confidentiality because the output is unreadable.
- □ B. MD5 is safe for general password hashing as long as you do not use a salt.
- □ C. If an attacker gets access to the database of hashed passwords, they cannot reverse the digests to recover the plaintext.
- □ D. Hashing is a reversible process if you possess the master decryption key.

(Select ALL that apply.)

4. What is the term for the situation when two different inputs to a hash function produce the exact same digest output? (1 pt)

5.3.b · Symmetric encryption and key management

5. A developer encrypts a bitmap image using AES in ECB mode. Even though AES itself is uncompromised, the ciphertext still reveals the outline of the original image. Why? (1 pt)

- ○ A. ECB mode encrypts identical plaintext blocks into identical ciphertext blocks, leaking patterns in the data

○ B. AES-128 keys are too short to hide image data

○ C. Bitmap files cannot be encrypted with any block cipher

○ D. ECB mode does not actually encrypt the first block

6. Select ALL statements that correctly describe Galois/Counter Mode (GCM). (3 pts)

□ A. GCM is an Authenticated Encryption with Associated Data (AEAD) mode

□ B. GCM provides both confidentiality and integrity verification via an authentication tag

□ C. GCM is functionally identical to ECB mode with a different name

□ D. If an attacker tampers with GCM ciphertext, decryption fails

(Select ALL that apply.)

7. A DEK (Data Encryption Key) needs to be stored long-term alongside the encrypted data it protects. Per the key-management lifecycle, what should be done with the DEK before storing it? (1 pt)

○ A. Store it in plaintext next to the ciphertext for fast access

○ B. Wrap (encrypt) it with a Key Encryption Key (KEK) kept in an HSM or secrets manager, and store the wrapped DEK

○ C. Hardcode it into the application source code

○ D. Email it to the database administrator for safekeeping

8. Name the type of random number generator that MUST be used to generate a symmetric key, instead of a standard library like Python's `random` module. (1 pt)

5.3.c · Integrity and authenticity (HMAC)

9. An attacker intercepts a message and its plain SHA-256 hash in transit. They change the message and recompute the hash over the new content before forwarding both to the recipient. What happens when the recipient checks integrity? (1 pt)

○ A. The check passes cleanly, because the attacker can compute a correct hash over any message they choose

○ B. The check fails, because SHA-256 detects tampering automatically

○ C. The check fails, because hash functions require a secret key to compute

○ D. The recipient cannot compute a hash at all without the original message

10. What is the key structural weakness that makes naive `H(key || message)` vulnerable to a length-extension attack on a Merkle-Damgard hash like SHA-256? (1 pt)

○ A. The digest exposes the hash function's internal state, letting an attacker resume hashing and append attacker-chosen data without knowing the key

○ B. The hash function is too slow to compute over long messages

○ C. SHA-256 always produces the same digest for every input

○ D. Concatenating the key and message causes a buffer overflow

11. Select ALL statements that correctly describe HMAC. (3 pts)

□ A. HMAC nests the hash function twice, folding the key into both an inner and outer keyed input

□ B. HMAC is standardized in NIST FIPS 198-1

□ C. HMAC is provably resistant to length-extension attacks even when built on SHA-256

□ D. HMAC encrypts the message so only the key holder can read its contents

(Select ALL that apply.)

12. Give the one-word term for the property that a plain hash provides (data was not altered) but does NOT prove on its own: that data really came from a specific, key-holding sender. (1 pt)

Coding Assignments (Python 3)

5.3.a · Encryption vs. hashing

3 pts · auto-graded on the site

Coding problem, 5.3.a: Encryption vs. hashing

Skill: Mitigate Risk · **Maps to:** 5.3.A · **Language:** Python 3 (pytest)

Problem

In secure systems, credentials like passwords should never be stored in their original plaintext form, nor should they be encrypted (which implies they could be decrypted if the key is compromised). Instead, they should be hashed using a one-way cryptographic function combined with a unique, random value called a **salt**. The salt ensures that identical passwords result in different digests, defeating precomputed lookup attacks (like rainbow tables).

Your task is to implement a password hashing and verification module.

You must implement the function `solve(action: str, password: str, param: str)` to handle two actions: 1. **Hash a password:** Given action `"hash"`, a plaintext password, and a salt, compute the SHA-256 hash of the concatenated string `salt + password`. Return a combined string of the format `salt$digest` (where `digest` is the hex-encoded hash). 2. **Verify a password:** Given action `"verify"`, a plaintext password, and a stored hash string in the format `salt$digest`, extract the salt and digest, compute the salted hash of the password, and verify if it matches the expected digest. Use a constant-time comparison helper like `hmac.compare_digest` to prevent timing side-channel attacks.

Function contract

Implement `solve(action: str, password: str, param: str) -> Union[str, bool]` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `action`: Either `"hash"` or `"verify"`
- `password`: The plaintext password
- `param`: Either the `salt` (if hashing) or the `stored_hash` of format `salt$digest` (if verifying)

Output

- If hashing: A string of format `salt$digest`
- If verifying: A boolean indicating whether the password matches the digest

Examples

```
solve("hash", "myPassword", "somesalt")
# -> "somesalt$4f24ef3001859bf9016e7a750ef90333d833890ea70c99a6cfbf6e568ea46a9a"
```

```
solve("verify", "myPassword", "somesalt$4f24ef3001859bf9016e7a750ef90333d833890ea70c99a6cfbf6e568ea46a9a")
# -> True

solve("verify", "wrongPassword", "somesalt$4f24ef3001859bf9016e7a750ef90333d833890ea70c99a6cfbf6e568ea46a9a")
# -> False
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter stub for 5.3.a."""
from typing import Union


def solve(action: str, password: str, param: str) -> Union[str, bool]:
    """Perform salted hashing or verification of passwords."""
    raise NotImplementedError("replace with the working reference solution")
```

5.3.b · Symmetric encryption and key management

3 pts · auto-graded on the site

Coding problem, 5.3.b: Symmetric key-management toolkit

Skill: Mitigate Risk · **Maps to:** 5.3.A, 5.3.B · **Language:** Python 3 (pytest)

Problem

Build three small helpers a security engineer would use when reasoning about symmetric key management, without relying on any real crypto library (standard library only, this is a toy, pedagogical model, not production cryptography).

1. **Key wrapping (envelope encryption).** Implement `wrap_key(dek, kek)` and `unwrap_key(wrapped, kek)` using a repeating-XOR keystream: each byte of dek is XORed with the KEK byte at the same position modulo `len(kek)`. This models how a Key Encryption Key (KEK) wraps a Data Encryption Key (DEK) for storage (see atom.md, "Key wrapping (Envelope Encryption)"). Unwrapping with the same KEK must recover the original DEK exactly, since XOR is its own inverse. 2. **Weak-RNG detection.** Implement `looks_like_csprng(key_bytes)`, a heuristic that flags keys that could NOT plausibly have come from a CSPRNG because they show low entropy / obvious structure (e.g., a single repeated byte, a short repeating pattern, or a strictly monotonic/arithmetic byte sequence like `0,1,2,3...`). Real CSPRNG output has no such structure. 3. **ECB pattern-leakage detection.** Implement `detect_ecb_leakage(ciphertext_blocks)`, which takes a list of same-size `bytes` blocks (as an ECB-mode block cipher would produce) and returns `True` if any two blocks are identical, the tell-tale sign that ECB mode is leaking repeated plaintext patterns into the ciphertext (see atom.md, "Electronic Codebook (ECB) Mode (Insecure)").

Function contract

Implement all three functions in `starter.py` (see `interface.md` / `blackbox.yaml`):

```
def wrap_key(dek: bytes, kek: bytes) -> bytes
def unwrap_key(wrapped: bytes, kek: bytes) -> bytes
def looks_like_csprng(key_bytes: bytes) -> bool
def detect_ecb_leakage(ciphertext_blocks: list) -> bool
```

Input

- `wrap_key`/`unwrap_key`: `dek`/`wrapped` are `bytes` (may be empty); `kek` is non-empty `bytes`.

- `looks_like_csprng`: `key_bytes` is `bytes`, at least 1 byte.
- `detect_ecb_leakage`: `ciphertext_blocks` is `list[bytes]`, all blocks the same length.

Output

- `wrap_key`/`unwrap_key` return `bytes` the same length as their key argument.
- `looks_like_csprng` returns `bool`.
- `detect_ecb_leakage` returns `bool`.

Examples

```
wrap_key(b"\x01\x02\x03\x04", b"\xff\x00")
# -> bytes([0x01^0xff, 0x02^0x00, 0x03^0xff, 0x04^0x00]) == b"\xfe\x02\xfc\x04"

unwrap_key(wrap_key(dek, kek), kek) == dek   # always, for any non-empty kek

looks_like_csprng(bytes([5]*16))            -> False   # single repeated byte
looks_like_csprng(bytes(range(16)))         -> False   # arithmetic/monotonic sequence
looks_like_csprng(b"\x4e\x91\x0a\xd3\x77\xbb\x02\xf8")  -> True   # no obvious structure

detect_ecb_leakage([b"AAAAAAAA", b"BBBBBBBB", b"AAAAAAAA"]) -> True    # block 0 == block 2
detect_ecb_leakage([b"AAAAAAAA", b"BBBBBBBB", b"CCCCCCCC"]) -> False
```

Constraints

- Deterministic, offline, standard library only. No `import random`, no real crypto libraries (`cryptography`, `pycryptodome`, `hashlib`-based ciphers, etc.).
- `kek` is always non-empty for `wrap_key`/`unwrap_key`; `dek` may be empty (returns `b""`).
- `looks_like_csprng` receives at least 1 byte; treat any input shorter than 8 bytes as "too short to judge" and return `False` (not enough data to look random).
- `detect_ecb_leakage` receives zero or more equal-length `bytes` blocks.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 5.3.b. Student completes the TODOs.
This file intentionally FAILS the tests until implemented."""


def wrap_key(dek: bytes, kek: bytes) -> bytes:
    """Wrap a Data Encryption Key (DEK) with a Key Encryption Key (KEK) using a
    repeating-XOR keystream (toy envelope encryption — NOT real crypto).

    Args:
        dek: plaintext key to wrap (may be empty).
        kek: non-empty wrapping key.
    Returns:
        bytes of len(dek), where byte i = dek[i] XOR kek[i % len(kek)].
    """
    # TODO: XOR each byte of dek with the KEK byte at the same position, wrapping
```

```
    #       the KEK index around with modulo len(kek).
    raise NotImplementedError("implement wrap_key() — see prompt.md")


def unwrap_key(wrapped: bytes, kek: bytes) -> bytes:
    """Inverse of wrap_key(): recovers the original DEK given the same KEK.

    Args:
        wrapped: bytes produced by wrap_key(dek, kek).
        kek: the same non-empty KEK used to wrap.
    Returns:
        the original dek bytes.
    """
    # TODO: XOR is its own inverse — this can reuse the same logic as wrap_key.
    raise NotImplementedError("implement unwrap_key() — see prompt.md")


def looks_like_csprng(key_bytes: bytes) -> bool:
    """Heuristically flag whether key_bytes could plausibly have come from a
    CSPRNG, vs. showing obvious low-entropy structure (repeated byte, short
    repeating pattern, or an arithmetic/monotonic sequence).

    Args:
        key_bytes: candidate generated key.
    Returns:
        False if len(key_bytes) < 8 (too short to judge), or if the bytes show
        obvious structure. True otherwise.
    """
    # TODO: check length < 8 -> False.
    # TODO: check all-identical bytes -> False.
    # TODO: check for a short repeating pattern (period 1..len//2) that tiles
    #       the whole input -> False.
    # TODO: check for a constant-step arithmetic sequence (mod 256) -> False.
    # TODO: otherwise -> True.
    raise NotImplementedError("implement looks_like_csprng() — see prompt.md")


def detect_ecb_leakage(ciphertext_blocks: list) -> bool:
    """Detect ECB-mode pattern leakage: True if any two blocks are identical.

    Args:
        ciphertext_blocks: list[bytes], all the same length.
    Returns:
        True if any duplicate block exists, False otherwise (or if < 2 blocks).
    """
    # TODO: compare blocks pairwise (or use a set) to find any duplicate.
    raise NotImplementedError("implement detect_ecb_leakage() — see prompt.md")
```

5.3.c · Integrity and authenticity (HMAC)

3 pts · auto-graded on the site

Coding problem, 5.3.c: Implement HMAC-SHA256 by hand

Skill: Mitigate Risk · **Maps to:** 5.3.B, 5.3.C · **Language:** Python 3 (pytest)

Problem

An API needs to verify that requests were sent by a client holding a shared secret key, and that the request body wasn't tampered with in transit. Rather than calling Python's built-in `hmac` module, build the FIPS 198-1 HMAC construction yourself on top of `hashlib.sha256`, so you can see exactly how the ipad/opad key-nesting defeats length-extension attacks.

Function contract

Implement, in `starter.py`:

- `hmac_sha256(key: bytes, message: bytes) -> str`, return the HMAC-SHA256 tag as a lowercase 64-character hex string.
- `verify(key: bytes, message: bytes, tag: str) -> bool`, recompute the tag and compare it (case-insensitively) to the given `tag` using a constant-time comparison.
- `solve(data: dict) -> dict`, the blackbox entry point that dispatches to the two functions above (see `interface.md` / `blackbox.yaml`).

You must implement the ipad/opad key derivation and the double-nested hashing yourself. You may use `hashlib.sha256` only as the inner hash primitive `H()`, do not import the `hmac` module for the core logic.

The construction (FIPS 198-1)

```
HMAC(key, message) = H( (key XOR opad) || H( (key XOR ipad) || message ) )
```

- Block size for SHA-256 is 64 bytes.
- If `key` is longer than 64 bytes, replace it with `H(key)` first.
- Right-pad `key` with `0x00` bytes up to exactly 64 bytes -> call this `k0`.
- `ipad` = the byte `0x36` repeated 64 times; `opad` = the byte `0x5c` repeated 64 times.
- `inner_key = k0 XOR ipad`, `outer_key = k0 XOR opad` (byte-wise XOR).
- `inner_hash = H(inner_key || message)`
- final tag = `H(outer_key || inner_hash)`, hex-encoded.

Input / Output

- `hmac_sha256(key, message)`: `key: bytes`, `message: bytes` -> hex `str` tag.
- `verify(key, message, tag)`: `tag: str` (hex, any case) -> `bool`.
- `solve(data)`: `data = {"op": "tag", "key": "<hex>", "message": "<str>"}` -> `{"tag": "<hex>"}`, or `data = {"op": "verify", "key": "<hex>", "message": "<str>", "tag": "<hex>"}` -> `{"valid": <bool>}`.

Examples

```
hmac_sha256(b"secretkey", b"transfer $100 to acct 42")
  -> "5b1f9a2c...b3" (64 hex chars; must match Python's hmac.new(key, msg, sha256).hexdigest())

verify(b"secretkey", b"transfer $100 to acct 42", "5b1f9a2c...b3") -> True
verify(b"secretkey", b"transfer $900 to acct 42", "5b1f9a2c...b3") -> False  # tampered message
```

Constraints

- Deterministic, offline, standard library only (`hashlib` only, no `hmac` module).
- Correctness is checked against Python's own `hmac` module inside `tests.py` (used there only as an independent oracle, not as part of your implementation).

- Must pass all cases in `tests.py` (visible + hidden), including empty keys/messages, keys longer than the 64-byte block size, and case-insensitive tag comparison in `verify()`.

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 5.3.c. Student completes the TODOs.
This file intentionally FAILS the tests until implemented."""
import hashlib

BLOCK_SIZE = 64  # SHA-256 block size in bytes
IPAD = 0x36
OPAD = 0x5C


def _h(data: bytes) -> bytes:
    """The inner hash primitive H(). Only stdlib hashlib.sha256 is used here."""
    return hashlib.sha256(data).digest()


def _derive_key(key: bytes) -> bytes:
    """Normalize key to exactly BLOCK_SIZE bytes per FIPS 198-1:
    hash keys longer than the block size, right-pad shorter keys with zero bytes."""
    # TODO: if key is longer than BLOCK_SIZE, replace it with _h(key).
    #       Then right-pad with zero bytes (b"\\x00") up to BLOCK_SIZE.
    raise NotImplementedError("implement _derive_key() — see prompt.md")


def _xor_bytes(data: bytes, pad: int) -> bytes:
    # TODO: XOR every byte of `data` with the int `pad`; return as bytes.
    raise NotImplementedError("implement _xor_bytes() — see prompt.md")


def hmac_sha256(key: bytes, message: bytes) -> str:
    """Compute HMAC-SHA256(key, message) and return the tag as a lowercase hex string.

    HMAC(key, message) = H( (key XOR opad) || H( (key XOR ipad) || message ) )
    """
    # TODO: derive k0 with _derive_key(key); build inner_key = k0 XOR ipad and
    #       outer_key = k0 XOR opad; compute inner_hash = H(inner_key || message);
    #       return H(outer_key || inner_hash) as a hex string.
    raise NotImplementedError("implement hmac_sha256() — see prompt.md")


def _constant_time_equal(a: str, b: str) -> bool:
    """Compare two equal-or-unequal-length strings without short-circuiting on the
    first mismatch, to avoid leaking timing information about where tags differ."""
    # TODO: return False immediately if lengths differ. Otherwise OR together the
    #       XOR of every character pair and return whether the total is zero.
    raise NotImplementedError("implement _constant_time_equal() — see prompt.md")


def verify(key: bytes, message: bytes, tag: str) -> bool:
    """Return True iff `tag` is the correct HMAC-SHA256 tag for (key, message)."""
    # TODO: recompute the expected tag and compare using _constant_time_equal().
    raise NotImplementedError("implement verify() — see prompt.md")


def solve(data: dict) -> dict:
    """Blackbox entry point. Dispatches on data['op']:
```

```
  - {"op": "tag", "key": <hex str>, "message": <str>}
        -> {"tag": <hex str>}
  - {"op": "verify", "key": <hex str>, "message": <str>, "tag": <hex str>}
        -> {"valid": <bool>}
Keys and messages are given as UTF-8 strings; `key` is provided as a hex string
to allow arbitrary binary-looking keys through a JSON-friendly interface.
"""
# TODO: decode data["key"] with bytes.fromhex, encode data["message"] as utf-8,
#       then dispatch to hmac_sha256() or verify() depending on data["op"].
raise NotImplementedError("implement solve() — see prompt.md")
```